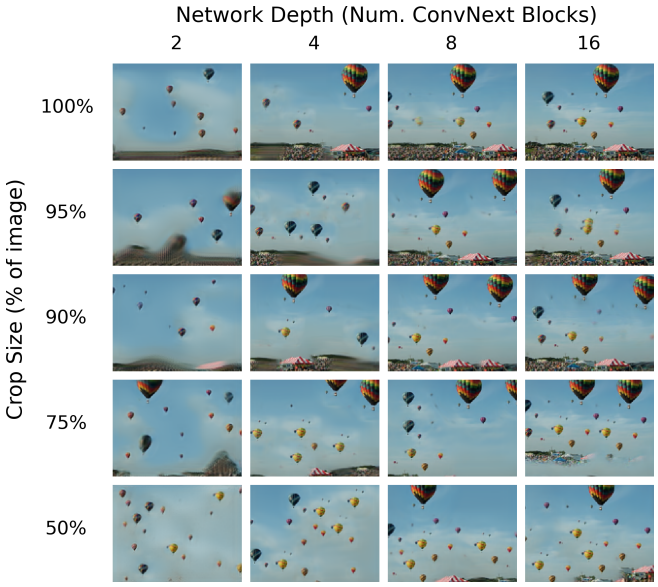
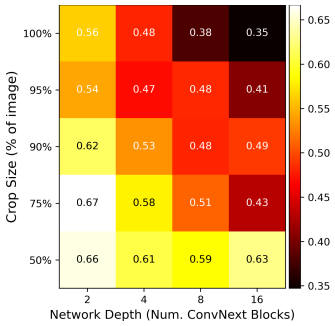
SinFusion: Training Diffusion Models on a Single Image or Video Supplementary Material

Anonymous CVPR submission

Paper ID 3831



(a) Quality



(b) NNF Diversity (See Sec.7 in main paper)

Figure A2. Analysing the effect of Crop-Size and Network-Depth on the diversity and quality of the generated outputs

A. Further Evaluations

A.1. Effect of Crop-Size and Receptive-Field

Our goal is to generate outputs (image / video) that preserve global structure, are of high quality, and with large diversity. These are affected by the choice of the crop-size on which the model is trained, and the effective receptive field of the model (determined by the depth of the convolutional model and controlled via the number of ConvNext blocks in the network). As seen in Fig. A2b, the largest diversity is achieved for small crop-size and small receptive field. However, small networks fail to learn the underlying image structure and result in blurry outputs (Fig. A2a). We therefore use more blocks for the model. This reduces the diversity, but dramatically improves outputs quality (as is evident from Fig. A2a). We choose the crop-size as a trade-off to preserve global-structure but also high diversity, which means using crop-size of about 95% of the image, with network depth of 16 blocks.

A.2. SinGAN [12] diversity score is easily fooled

Quantifying the "diversity" of generated outputs from a single image or video is hard. To begin with, other than the most simple case of generating the same input image over and over again, which will have a diversity score of zero, there's no definition for "diversity" other than a subjective human factor, looking on a set of images and deciding whether it is diverse or not. SinGAN [12] proposed to calculate the standard deviation of the intensity values of each pixel over all generated samples, then average this over all the pixels, and then divide the result by the standard deviation of the intensity values of the original image. This method aligns with the simple case of generating only one sample described above, but will fail to obtain a valid score for the following simple example: given an input image, generate "new" samples by applying random translations to the image. This will generate many *different* samples, therefore not having a diversity of "zero", but it is reasonable to

assume that most people will not find such a generated set as diverse. In fact, it is very easy to achieve a high "diversity score" by sampling once a random image, which will act as the input image, then generate "new samples" by random translations (in this case the proposed diversity score will converge to 1).

In Tab. A1 we add to the original Table 1 from the main paper (colored in blue here), the SinGAN-diversity scores of all methods, plus the SinGAN-diversity score for the *tranlsated noise*, and as can be seen, it is indeed very close to 1. We also add the other scores for comparison.

As opposed to this high scores given by SinGAN-diversity, note how our NNFDIV diversity score gives a very low score for the translated noise example.

Dataset	Method	NNFDIV ↑	NNFDIST ↓	SVFID↓	DIV [12]
SinGAN-GIF	VGPNN[3]	0.20	0.28	0.0058	0.60
	SinGAN-GIF [1]	0.40	1.10	0.0119	0.86
	SinFusion (Ours)	0.30	0.45	0.0090	0.65
HP-VAE-GAN	VGPNN[3]	0.22	0.14	0.0072	0.45
	HP-VAE-GAN [2]	0.31	0.39	0.0081	0.41
	SinFusion (Ours)	0.35	0.26	0.0107	0.44
	Translated Noise	0.06	$6.8 \cdot 10^{-4}$	$1.1 \cdot 10^{-7}$	0.96

Table A1. A simple "translated noise" receives a high score of ~ 1 by SinGAN[12] proposed diversity score, while our NNFDIV manages to better handle this example, giving it a very low diversity score (blue parts are copied from main paper for comparison).

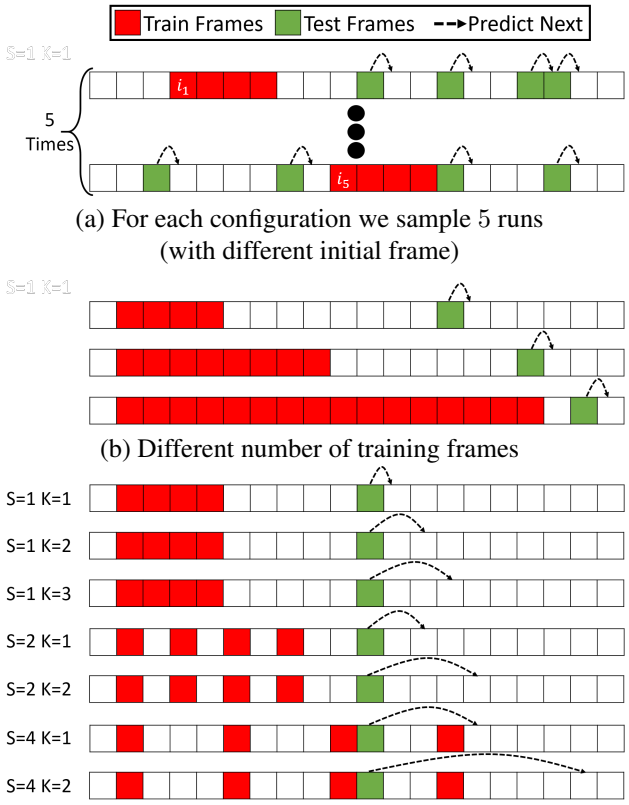
A.3. Further Explaining Fig.8 Experiment

We further elaborate on the specific details of the experiment whose results appear in Fig.8 in the main paper.

We are performing *frame prediction from a single video*. Each "run" involves training a model on n frames. After training, 100 frames (not seen during training) are chosen, and the model predicts the next frame for each of them. The total score for each run is the average MSE between predicted and ground-truth frames. Here we explain the difference between each "run":

Fig.8a: Number of Training Frames For each number of training frames n (where $n = [4, 8, 16, 32, 64]$) we sample at random 5 starting points $i_1, ..., i_5$. For each starting point we use the n frames starting from frame i as the training samples. This is depicted in Fig. A3a – training frames (red) and test frames (green), where each test frame is used to predict the next frame. In Fig. A3b we show the schemes for runs trained on different number of frames.

Fig.8b: Video "speed" and k In Fig.8b (in the main paper) we fix the number of training frame to 32. For speed=1, this is similar to the same experiment as shown in Fig.8a for $n = 32$. For speed=2, for each random starting frame, we choose the next 32 frames of the $1/2$-subsampled video,



(a) For each configuration we sample 5 runs (with different initial frame)

(b) Different number of training frames

(c) Different choices of video-speed (S) and k

Figure A3. **Frame Prediction from a Single-Video**. Depicting evaluations experiments from Sec.7/Fig.8 in main paper

e.g., if the starting point is frame number 17, the training frames will be frames number $17, 19, 21, ..., 79$. For speed=4 and starting point 17 the training frames will be frames number $17, 21, 25, ..., 141$. The parameter k is the same as in the training procedure of the DDPM-Frame-Predictor, namely, it is the frame difference between current frame and the predicted frame. That is, if $k = 1$, for each test frame number n we will predict the frame $n + 1$; if $k = 2$ we will predict $n + 2$ etc. Note that if the video is already subsampled, k is the frame difference w.r.t the subsampled video. In Fig. A3c we depict several choices for the speed (denoted by S) and k.

B. Comparisons

B.1. Generation from Single-Video (Table 1)

We run our comparisons on the data provided by the previous works on video generation from a single video: VGPNN [3], HP-VAE-GAN [2] and SinGAN-GIF [1]. We follow the same methodology used in VGPNN [3].

We compare to two datasets of videos. One provided by SinGAN-GIF [1] and the other by HP-VAE-GAN [2].

In SinGAN-GIF there are 5 videos with 8 to 15 frames, each of maximal spatial resolution 168×298. For each of the 5 input videos, each method generates 6 samples. In HP-VAE-GAN there are 10 videos each of spatial resolution 144×256. For each of the 10 input videos, each method generates 10 samples. HP-VAE-GAN and VGPNN only use the first 13 frames since their methods are limited by runtime and memory (as explained in the main paper). Since learning on small amounts of data is not a goal for the task of diverse generation from a single video, and since our framework can easily learn from much more data, we train our framework on longer sequences of frames from the given input videos.

B.2. Comparison to VDM [6]

In the supplementary HTML we show the results of Video-Diffusion-Models (VDM) [6] trained on a single video. Since the official implementation was not published at the time of writing, we use a third-party implementation [1]). Since VDM expects a dataset of videos, we slice a long video of 420 frames into 42 short videos of 10 frames each and let VDM train on those videos. We could only train the model on a resolution of 64×64 pixels before exceeding the memory of our GPU. We trained the model for about 150 epochs using two different learning rates (total run time was about a day on a V100 GPU). The results seems to capture the motions of the original videos, but it is difficult to evaluate since the resolution is too low compared to the original videos. The results also consists of artifacts that may result from the low amount of data usually needed to train such models (without including our proposed modifications in the main paper). It is qualitatively evident from the results that our framework, SinFusion, generates outputs of much higher quality when trained on a single video. For video results please see the accompanying HTML files.

C. Ablations

Predicting Image Instead of Noise. As opposed to the standard DDPM [5] training, our single-image-DDPM model outputs a prediction for the un-noised input crop. In Fig. A4 we show examples for our generated outputs for predicting the crop/image (top) against generated outputs for predicting noise (bottom). As shown, predicting the un-noised crop instead of the noise generates higher quality images. It also does so with much fewer training iterations.

Architectural changes. We check the importance of our proposed architectural modifications to the original DDPM [5] by reverting each change and generating several images. We compare these generated images to the images generated by our final model. An example for these comparisons can be seen in Fig. A5.
The first comparison shows outputs of our model with upsampling and downsampling layers. The generated outputs completely overfit the training image, and have no diversity.
The second comparison shows outputs of our model with attention layers. Other than significantly increasing training time (to almost 2 hours per training image) and generation time, the added attention doesn't improve the generated image quality.
The third comparison shows outputs of our model with all ConvNext [8] blocks replaced with ResNet [4] blocks. The generated outputs suffer from more artifacts and are of lesser quality than our generated outputs.

Importance of DDPM frame Projector in diverse video generation. We show the necessity of our DDPM frame Projector model as part of the diverse video generation framework. In this ablation, we generate videos from several input videos using only the DDPM frame Predictor to generate frames, without using the Projector model to correct small artifacts in the generated frames. In all examples, it can be seen that the small artifacts, which remain uncorrected, accumulate over time and severely degrade the generation quality. For video results please see the accompanying HTML files.

D. Implementation Details

Our code is implemented with PyTorch [10]. We make the following hyper-parameters choices:

- We use a batch size of 1. Each large crop contains many large "patches". Since our network is a fully convolutional network, each large "patch" is a single training example.
- We use ADAM optimizer [7] with a learning rate of 2×10^{-4}, reduced to 2×10^{-5} after $100K$ iterations.
- We set the diffusion timesteps $T = 50$. This allows for fast sampling, without sacrificing image/video quality (This trade-off is simpler in our case because of the simplicity of our learned data distribution).
- When generating diverse videos, we use the DDPM frame Projector to correct predicted frames by noising and denoising $T_{corr} = 3$ steps.
- We compared several noise schedules for the diffusion models and ended up using linear noise schedule ($\beta_0 = 2 \times 10^{-3}, \beta_T = 0.4$) for single-image DDPM and cosine noise schedule [9] for single-video DDPM.

[1] https://github.com/lucidrains/video-diffusion-pytorch

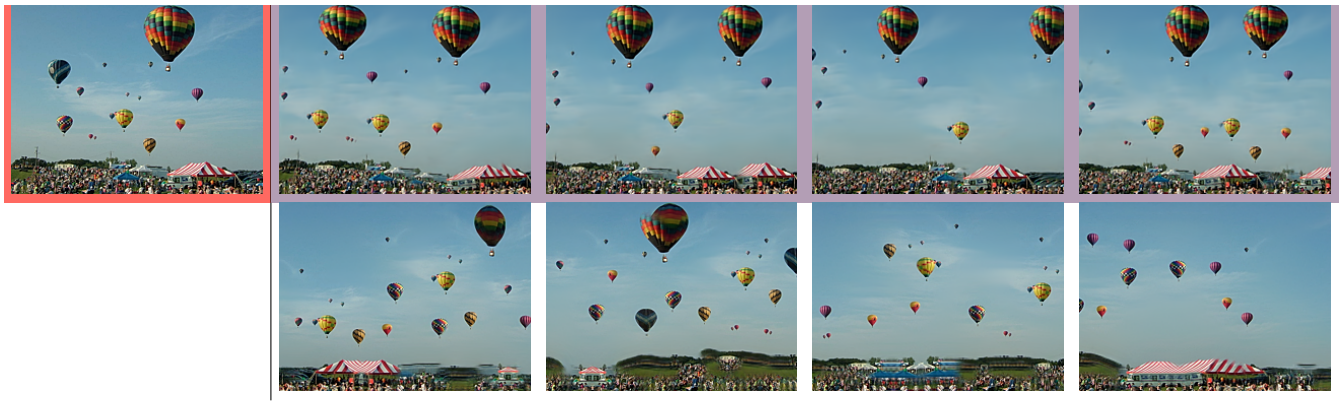


Figure A4. **Noise vs Image prediction ablation**. *Top Row*: Input image (left;red) and generated outputs of our final model (right;purple) – predicts the un-noised image crop. *Bottom Row*: Generated outputs using the standard DDPM noise prediction.

Figure A5. **Architectural ablations**. *Top Row*: Input image (left;red) and Generated outputs of our final model (right;purple).
2nd Row: Generated outputs of our model with upsampling and downsampling layers.
3rd Row: Generated outputs of our model with added attention layers (similar to the standard DDPM [5] Unet[11] network).
4th Row: Generated outputs of our model where each ConvNext [8] block is replaced with a ResNet [4] block.

- Our standard network architecture consists of 16 ConvNext [8] blocks, each block with a base dimension of 64 channels.

D.1. Runtimes

On a Tesla V100-PCIE-16GB, for images/videos of resolution 144×256, our model trains for about 1.5 minutes per 1000 iterations, where each iteration is running one diffusion step on a large image crop. The total amount of iterations and total runtime for each of our models are:

- Single-Image DDPM - $50K$ iterations, total runtime of 80 minutes (good results are already seen after $15K$ iterations).
- Single-Video DDPM Frame Predictor - $200K$ iterations, total runtime of 5.5 hours.
- Single-Video DDPM Frame Projector - $100K$ iterations, total runtime of 2.5 hours
- Single-Video DDPM Frame Interpolator - $50K$ iterations, total runtime of 1.5 hours.

References

[1] Rajat Arora and Yong Jae Lee. Singan-gif: Learning a generative video model from a single gif. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1310–1319, 2021. 2

[2] Shir Gur, Sagie Benaim, and Lior Wolf. Hierarchical patch vae-gan: Generating diverse videos from a single sample. *arXiv preprint arXiv:2006.12226*, 2020. 2

[3] Niv Haim, Ben Feinstein, Niv Granot, Assaf Shocher, Shai Bagon, Tali Dekel, and Michal Irani. Diverse generation from a single video made possible. *arXiv preprint arXiv:2109.08591*, 2021. 2

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 3, 4

[5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020. 3, 4

[6] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *arXiv preprint arXiv:2204.03458*, 2022. 3

[7] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 3

[8] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022. 3, 4

[9] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 3

[10] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017. 3

[11] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015. 4

[12] Tamar Rott Shaham, Tali Dekel, and Tomer Michaeli. Singan: Learning a generative model from a single natural image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4570–4580, 2019. 1, 2